

ARTIFICIAL INTELLIGENCE · ADVANCED

Forward Deployed AI Engineering

Move from AI experiments to dependable systems.



Full-Stack / Backend Developer -> GenAI Developer -> Production AI Engineer -> AI FDE.



8-12 weeks



6 modules



Professional learning

[Explore the learning experience →](#)

Practical learning. Clear progression. Work you can explain.

YOUR NEXT CAPABILITY

What will you be able to do?

A technology-agnostic, practice-first progression for engineers who build AI applications, productionize them, integrate enterprise systems and own measurable outcomes.



Build LLM, RAG, agent, voice and multimodal applications



Operate AI with evaluation, guardrails, security and observability



Integrate identity, APIs, databases and enterprise systems



Own discovery, adoption and measurable business outcomes



Who this is for

- Python developers
- C#/.NET developers
- Java/Spring developers
- Node.js/TypeScript developers
- Tech leads and architects



Your starting point

- Backend programming experience
- Working knowledge of APIs, JSON, Git and databases
- Python is helpful but not mandatory

Your progression at a glance

FROM CONCEPT TO EVIDENCE



Learn it. Apply it. Explain it.

Move from guided concepts to practical exercises and a coherent piece of work. Use the course resources to revisit difficult ideas and make your reasoning visible.

THE CURRICULUM

A clear route through the subject.

Every module builds towards practical work. The course outline below is aligned with the academy's current program catalog.

1 Engineering Readiness Bridge

Optional readiness assessment and bridge.

- Backend/API readiness
- Data and database readiness
- Testing and Git
- Cloud foundations

2 AI Engineering

LLMs, context, structured outputs, embeddings and RAG.

- LLM foundations
- Prompt and context engineering
- Structured output and tools
- Embeddings, vector databases and RAG

3 Agents, Voice and Multimodal AI

Agentic and multimodal application patterns.

- Agents, tasks and memory
- MCP, skills and multi-agent patterns
- STT, TTS and realtime agents
- Vision and document AI

4 Production AI Engineering

Reliable, secure and observable AI delivery.

- Evaluations and guardrails
- Docker, CI/CD and cloud
- LLMOps and observability
- Security, scale and cost

5 Enterprise Integration

Identity and systems-of-record integration.

- SSO, OAuth/OIDC and RBAC
- Enterprise APIs and databases
- SharePoint, SAP and CRM integration
- Service platforms and legacy systems

6 Solution Architecture and Forward Deployment

Choose the right pattern and own outcomes.

- RAG vs fine-tuning vs SLM
- Agent vs deterministic workflow
- Customer discovery and value case
- Forward-deployment capstone

Practice with purpose

Frame a real information need, connect permitted sources, build a grounded assistant and present an evaluation and human-review plan.

BUILD SOMETHING THAT DEMONSTRATES YOUR LEARNING

Enterprise knowledge assistant

Frame a real information need, connect permitted sources, build a grounded assistant and present an evaluation and human-review plan.

**Task and integration map****Grounded response examples****Evaluation and operational runbook**

Tools and working methods

LLM APIs

Vector search and RAG

Agents, MCP and skills

Docker and cloud

Evaluation and observability

Tools are part of the learning workflow, not partner endorsements. Examples may evolve while preserving the learning outcomes.



Delivery

Hybrid learning · 8–12 weeks

Confirm the current schedule, delivery arrangements and any prerequisites before enrolling.



Completion

A completion certificate is available after the required course work and a passing assessment.

Course completion is not a job, placement or funding guarantee.

Your next steps

- 1 Review the curriculum and your starting point.
- 2 Review current enrollment options and delivery details.
- 3 Start learning, keep your evidence and track your progress.

[Open the course experience →](#)

Customer-preview edition. Interactive links open the local academy prototype. Confirm course availability, schedule and commercial terms with the academy.

INSIDE THE LEARNING EXPERIENCE

Design an evidence-grounded service assistant

An internal service team needs answers from approved policy documents and ticket summaries. Some documents are restricted, and some requests ask the system to take an action.

Understand the concept

Separate retrieval, answer generation and action execution. Apply the requesting user's permissions before retrieved text reaches the model. Attach source identifiers so an answer can point to its evidence. Treat document instructions as untrusted content. For a state-changing tool, validate the arguments and require the appropriate approval; a generated sentence is not authorization.

Worked example

Request + user identity

- > permission-filtered retrieval
- > evidence selection with source IDs
- > grounded answer or insufficient-evidence response
- > proposed action (if requested)
- > schema validation + permission + human approval
- > execution receipt + audit event

Interpret the result

A useful answer includes evidence and a clear limit when sources disagree or are insufficient. A proposed ticket update remains a proposal until the authorized action boundary approves it. The same architecture can be implemented with Python, .NET, Java or Node.js.

Knowledge check

Why is retrieval quality alone insufficient for a production assistant?

Explanation: Permissions, evidence handling, tool authorization, failure recovery and operational monitoring also determine whether the system behaves safely and usefully.

YOUR PRACTICAL ASSIGNMENT

Build evidence, not just familiarity.

Design an evidence-grounded service assistant



Your task

- Create an evaluation set with answerable, unanswerable and restricted-source requests.
- Implement a read-only adapter for an enterprise-system fixture.
- Demonstrate a failed retrieval, a denied action and a successful approved action.



Submission review criteria

- No restricted evidence reaches an unauthorized request
- Answers cite supporting source IDs and abstain when needed
- Tool calls validate inputs and produce auditable execution outcomes

Submit

A working artifact or analysis, a short explanation of your decisions, and evidence that addresses each review criterion.

Reflect

Describe one failed approach, how you diagnosed it, and what you would test next. Identify limitations instead of hiding them.

Your project connection

Frame a real information need, connect permitted sources, build a grounded assistant and present an evaluation and human-review plan.

Use the sample assignment as a stepping stone toward the course project, not as a substitute for the full curriculum.

[Explore the worked lesson](#) →

Proposed teaching sample for curriculum and UX review. Full lesson production, instructor review and delivery scheduling remain separate work.